

# Problem Set 1 — Answers

Roy Sarafov 209619477

## Question 1 — 2D Parity Error Correction Code

### 1.1

We add one parity bit per row and one per column, giving  $k + l$  parity bits total.

To minimize  $k + l$  subject to  $k \times l = n$ , by AM-GM:  $k + l \geq 2\sqrt{k \cdot l} = 2\sqrt{n}$ , with equality when  $k = l = \sqrt{n}$ . So the minimum number of parity bits is  $\lceil 2\sqrt{n} \rceil$ , achieved with a square matrix.

For general  $n$  (not a perfect square), we pick the integer pair  $(k, l)$  with  $k \cdot l = n$  that minimizes  $k + l$ . For example,  $n = 12$ : candidates are  $(1, 12) \rightarrow 13$ ,  $(2, 6) \rightarrow 8$ ,  $(3, 4) \rightarrow 7$ , so the optimal is a  $3 \times 4$  matrix with 7 parity bits.

### 1.2

The minimum number of errors for which an undetectable pattern exists is 4. That is, every pattern of 1, 2, or 3 errors is always detected, but there exists at least one 4-error pattern that is not.

Proof:

- 1–3 errors: Any 1, 2, or 3-bit error must violate at least one row or column parity  $\rightarrow$  always detected.
- 4 errors — undetectable case exists: Flip the 4 corners of any rectangle in the matrix: positions  $(r1, c1)$ ,  $(r1, c2)$ ,  $(r2, c1)$ ,  $(r2, c2)$ . Each affected row and column get exactly 2 flips  $\rightarrow$  all parities remain even  $\rightarrow$  not detected.

Note: many 4, 5, 6, 7-error patterns are still detected — but since this one 4-error pattern escapes detection, 4 is the minimum.

### 1.3

The extra global parity bit does not improve the code.

The only undetectable error patterns in the 2D parity code are those that form a “rectangle” — flipping an even number of bits in each affected row and column. All such patterns flip an even total number of bits, so the global parity bit remains unchanged, and the error still goes undetected.

Since every pattern the 2D code misses also evades the global parity bit, the addition provides no improvement in error detection.

## Question 2 — Internet Checksum

### 2.1

cksum computes the ones-complement sum of all 16-bit words in the buffer. The sender appends the checksum to the message. The receiver sums all words including the checksum — if no errors occurred, the result is 0xFFFF. Any deviation means an error was detected.

A single-bit flip changes exactly one word by  $2^i$  (for some  $0 \leq i \leq 15$ ), which changes the total ones-complement sum by exactly  $2^i \neq 0$ . Therefore, the receiver's sum will differ from 0xFFFF  $\rightarrow$  error detected.

Example: message = 0x1111, 0x2222, checksum = 0xCCCC. - No error:  $0x1111 + 0x2222 + 0xCCCC = 0xFFFF \checkmark$  - Bit 0 flipped in first word ( $0x1111 \rightarrow 0x1110$ ):  $0x1110 + 0x2222 + 0xCCCC = 0xFFFE \neq 0xFFFF \rightarrow$  detected  $\checkmark$

### 2.2

We need to find two-bit flips that leave the ones-complement sum unchanged.

Construction: Flip bit  $i$  in one word and bit  $i$  in another word — one flip adds  $2^i$  to the sum, the other subtracts  $2^i \rightarrow$  net change = 0  $\rightarrow$  undetected.

Concrete example: - Message: 0x1111, 0x2222, checksum = 0xCCCC - Flip bit 0 in first word:  $0x1111 \rightarrow 0x1110$  (-1) - Flip bit 0 in second word:  $0x2222 \rightarrow 0x2223$  (+1) - Receiver sum:  $0x1110 + 0x2223 + 0xCCCC = 0xFFFF \rightarrow$  no error detected  $\times$

Two bits flipped, completely undetected. ■

### 2.3

A single parity bit operates over all bits as a single unit — it only detects an odd number of flipped bits.

cksum operates on 16-bit words and accumulates their values, so:

1. Magnitude matters — it detects errors that change the *value* of a word, not just its parity. Flipping two bits in the same word that cancel in parity is missed by a parity bit but caught by cksum if the net value change is nonzero.
2. Burst errors — a burst error within a single 16-bit word changes its value, which cksum detects. A single parity bit only catches odd-length bursts.
3. Practical error patterns — real network errors tend to be burst errors or affect specific bit positions. cksum catches a much wider range of these than a single parity bit.

In short: a single parity bit adds 1 bit of redundancy and catches very little. cksum adds 16 bits of redundancy and catches far more real-world error patterns. Additionally, a single

parity bit has a 1/2 chance of accidentally matching corrupted data, while the checksum reduces this to 1/65536.

## Question 3 — Hamming Code (n, k, 3)

### 3.1

The standard Hamming code has minimum distance  $d = 3$ , which means it can correct 1-bit errors — the  $m$  parity bits form a binary “coordinate” that points to exactly which bit flipped. However, with  $d=3$  alone, the code cannot simultaneously correct 1-bit errors and detect 2-bit errors: if 2 bits flip, the syndrome still points to some position and the decoder may miscorrect, making things worse.

Proof by counting (perfect code argument):

- Total number of bit strings of length  $n = 2^n$ .
- Number of codewords  $= 2^k = 2^{2^m-1-m}$ .
- Each codeword has a Hamming sphere of radius 1 containing  $1 + n = 1 + (2^m - 1) = 2^m$  strings (the codeword itself + all strings at distance 1).
- Total strings covered  $= 2^k \times 2^m = 2^{2^m-1-m} \times 2^m = 2^{2^m-1} = 2^n$ . ✓

The spheres cover exactly all  $2^n$  strings with no gaps and no overlaps (since  $d=3$  means spheres of radius 1 are disjoint). Therefore, every bit string of length  $n$  lies within distance 1 of exactly one codeword. ■

### 3.2

The extra global parity bit does improve the code.

The standard Hamming code ( $d=3$ ) can correct 1-bit errors but cannot distinguish a 1-bit error from a 2-bit error — both produce a non-zero syndrome pointing to some bit position, and the decoder may miscorrect a 2-bit error.

Adding a global parity bit raises the minimum distance to  $d=4$ , which allows the decoder to distinguish the two cases:

- 1-bit error: syndrome points to a position + global parity is odd → correct it.
- 2-bit error: syndrome points to a position + global parity is even → known to be a 2-bit error, do not correct, report error.

So, the improvement is: the code can now correct 1-bit errors AND detect 2-bit errors simultaneously, without the risk of mis correction.

Note the contrast with Q1.3: there, the 2D parity code’s minimum undetectable pattern (4-corner rectangle) already flipped an even number of bits, so the global parity gave no new information. Here, the Hamming code’s minimum distance is odd ( $d=3$ ), so adding a parity bit genuinely increases it to  $d=4$ .

## Question 4 — Limits of Error Detection with Redundancy

### 4.1

We show no code with 2 redundancy bits can detect all  $\leq 2$ -bit errors on 5-bit messages.

Proof (by pigeonhole):

Messages of Hamming weight 1 over 5 bits: 10000, 01000, 00100, 00010, 00001 — there are 5 such messages.

Each gets 2 redundancy bits  $\rightarrow$  only 4 possible redundancy patterns (00, 01, 10, 11).

By pigeonhole: 5 messages, 4 patterns  $\rightarrow$  at least two messages share the same redundancy bits. Call them M1 and M2.

Any two weight-1 messages differ in exactly 2 positions, so a 2-bit error can transform M1 into M2. The receiver sees M2 with its correct redundancy  $\rightarrow$  looks valid  $\rightarrow$  error undetected.

Concrete example:

Suppose 10000 and 01000 are both assigned redundancy bits 01.

- Sender transmits M1 = 10000|01
- 2-bit error: bit 0 flips (1 $\rightarrow$ 0), bit 1 flips (0 $\rightarrow$ 1)  $\rightarrow$  received: 01000|01
- Receiver checks 01000 with redundancy 01 — matches M2's valid encoding  $\rightarrow$  error not detected X

### 4.2

We use the same pigeonhole argument as 4.1.

A code with  $r=32$  redundancy bits can detect all  $\leq 8$ -bit errors only if all messages of Hamming weight 8 have distinct redundancy patterns. There are  $2^{32}$  possible patterns, so we need  $\binom{N}{8} \leq 2^{32}$ . When  $\binom{N}{8} > 2^{32}$ , two weight-8 messages must share the same redundancy bits, and an 8-bit error can transform one into the other undetected.

Finding smallest N where  $\binom{N}{8} > 2^{32} = 4,294,967,296$ :

- $\binom{63}{8} = 3,872,894,697 < 2^{32}$  X
- $\binom{64}{8} = 4,426,165,368 > 2^{32}$  ✓

N = 64

### 4.3

For correction, the Hamming (sphere-packing) bound applies. Each codeword requires a sphere of radius 8 (all strings within Hamming distance 8) that doesn't overlap with any

other sphere. With N-bit messages and 32 redundancy bits, the codeword length is N+32, and the bound requires:

$$\sum_{i=0}^8 \binom{N+32}{i} \leq 2^{32}$$

When this is violated, correction of all  $\leq 8$ -bit errors is impossible. Finding the smallest N where the sum exceeds  $2^{32}$ :

- N=30:  $\sum_{i=0}^8 \binom{62}{i} = 3,941,437,837 \leq 2^{32} \times$
- N=31:  $\sum_{i=0}^8 \binom{63}{i} = 4,501,777,129 > 2^{32} \checkmark$

N = 31

## Question 5 — M/M/1 Queue Splitting

Setup: Original M/M/1 system with arrival rate  $\lambda$  and service rate  $\mu$ , split into n subsystems proportionally to  $a_1, a_2, \dots, a_n$  ( $A = \sum a_i$ ). Subsystem i gets arrival rate  $\lambda_i = (a_i/A) \cdot \lambda$  and service rate  $\mu_i = (a_i/A) \cdot \mu$ .

### 5.1

Send each arriving customer to subsystem i with probability  $a_i/A$ , independently for each customer (probabilistic/weighted random routing).

This works by the Poisson splitting property: randomly routing customers from a Poisson( $\lambda$ ) stream with probability  $p_i$  produces independent Poisson( $p_i \cdot \lambda$ ) streams. Each subsystem therefore receives Poisson arrivals with exponential service and 1 server  $\rightarrow$  valid M/M/1 queue.

### 5.2

Subsystem i has arrival rate  $\lambda_i = (a_i/A) \cdot \lambda$  and service rate  $\mu_i = (a_i/A) \cdot \mu$ . The average waiting time in an M/M/1 queue is  $1/(\mu - \lambda)$ , so:

$$W_i = \frac{1}{\mu_i - \lambda_i} = \frac{1}{\frac{a_i}{A}(\mu - \lambda)} = \frac{A}{a_i(\mu - \lambda)}$$

Subsystems with smaller  $a_i$  have longer waiting times — the absolute rates are smaller so the queue drains more slowly.

### 5.3

The overall average waiting time is the weighted average across all subsystems, weighted by the fraction of customers each receives:

$$W = \sum_{i=1}^n \frac{a_i}{A} \cdot W_i = \sum_{i=1}^n \frac{a_i}{A} \cdot \frac{A}{a_i(\mu - \lambda)} = \sum_{i=1}^n \frac{1}{\mu - \lambda} = \frac{n}{\mu - \lambda}$$

The  $a_i$  cancel completely — the overall waiting time is always  $n/(\mu - \lambda)$ , regardless of how the split is chosen. Splitting into  $n$  subsystems always makes the average waiting time  $n$  times worse than the original system.

## 5.4 (Bonus)

The overall waiting time  $W = n/(\mu - \lambda)$  is constant — it does not depend on the values of  $a_1, a_2, \dots, a_n$  at all. Therefore there is no unique maximum; every valid split (any  $a_i \geq 0$  with  $A > 0$ ) achieves the same waiting time.

## Question 6 — Ethernet Capture Effect

Setup: After the first collision, A draws 0 and B draws T, so A transmits its 1st frame. Then both try again → 2nd collision. A picks from  $\{0, T\}$  uniformly, B picks from  $\{0, T, 2T, 3T\}$  uniformly.

### 6.1

Enumerate all  $2 \times 4 = 8$  equally likely (A,B) backoff combinations:

| A \ B | 0      | T      | 2T     | 3T     |
|-------|--------|--------|--------|--------|
| 0     | tie    | A wins | A wins | A wins |
| T     | B wins | tie    | A wins | A wins |

- A wins: 5 cases, B wins: 1 case, tie: 2 cases.

Option 1 (raw probability):  $P(\text{A wins}) = 5/8$

Option 2 (conditioning on a decisive outcome, ties re-draw):  $P(\text{A wins}) = 5/6 \approx 0.833$

### 6.2

After A wins frame  $i$ , its window resets to  $\{0, T\}$ , while B's window has grown to  $\{0, T, \dots, (2^{i+1} - 1)T\}$  (size  $2^{i+1}$ ). The probability that A wins frame  $i + 1$  is:

$$p_i = 1 - \frac{1}{2^{i+1}} \cdot \frac{2}{2} = \frac{2^{i+1} - 1}{2^{i+2}}$$

More precisely, A loses only when B picks 0 and A picks T, which has probability  $\frac{1}{2} \cdot \frac{1}{2^{i+1}} = \frac{1}{2^{i+2}}$ , so  $p_i = 1 - \frac{1}{2^{i+2}}$ .

The probability A wins all  $F$  frames is  $P = \prod_{i=1}^F p_i$ . Taking the log and using  $\ln(1 - x) \geq -\frac{x}{1-x} \approx -x - x^2$  for small  $x$ :

$$\ln P = \sum_{i=1}^F \ln \left( 1 - \frac{1}{2^{i+2}} \right) \geq - \sum_{i=1}^F \frac{1.5}{2^{i+2}}$$

As  $F \rightarrow \infty$  the geometric series sums to  $\frac{1.5}{4} \cdot \frac{1}{1-1/2} = \frac{1.5}{2} = 0.75$ , so  $\ln P \geq -0.75$ , giving  $P \geq e^{-0.75} \approx 0.47$ .

There is at least a ~47% chance that B never transmits a single frame while A sends all of its frames. This is the capture effect: once A gains the channel, its always-reset window gives it a structural advantage that compounds with every frame, effectively starving B indefinitely.